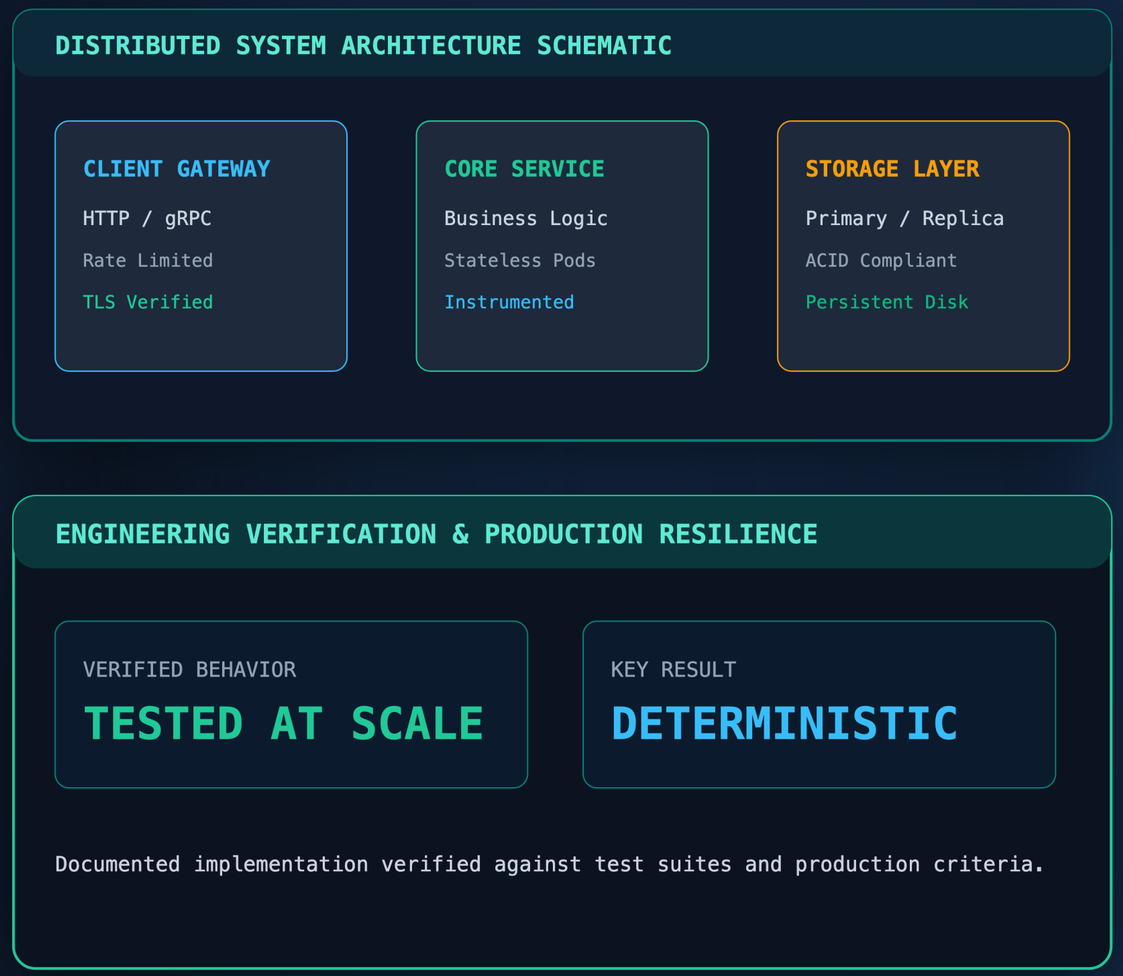
Software Engineering Lab

Lab 17

Architecture Decision Record

Parser & Linter untuk Validasi Struktural Keputusan Arsitektur



Lukman / lukman-ss

Konteks Keputusan Arsitektur Hilang Tanpa Jejak

Engineer baru bertanya mengapa sistem dibangun begini; tidak ada jawaban yang terekam.

MASALAH NYATA

Putaran architect merangkap code review: pertanyaan berulang 'kenapa kita pilih X?' di every PR.
Keputusan diambil via chat atau meeting, lalu tenggelam dalam commit history.
ADR lama tidak diperbarui saat sistem berubah; status dokumentasi drift dari kode aktual.

RISIKO BISNIS

Re-debate arsitektur lama memakan waktu engineering sprint tanpa solusi baru.
Keputusan yang salah diterapkan karena engineer tidak mengetahui ADR penggantian.

Dalam praktik engineering sehari-hari, keputusan arsitektur sering kali diambil secara informal—lewat chat, voice note, atau diskusi meeting. Ketika engineer baru bergabung atau sistem mengalami pivot strategis, konteks historis mengapa pilihan tertentu dibuat hilang. Tim kemudian terjebak debat berulang tentang keputusan yang sebenarnya sudah pernah diverifikasi dan ditolak. Ini bukan masalah dokumentasi biasa; ini adalah kehilangan kapabilitas keputusan yang berharga.

Architectural context without structural records becomes tribal knowledge.

Review Markdown Manual Lupa Relasi Penggantian

Engineer bisa saja update isi ADR lama tapi lupa tambahkan tautan supersesi ke ADR baru.

KEGAGALAN REVIEW MANUAL

ADR 1 tetap berstatus 'Accepted' padahal sudah diganti ADR 2 — lupa di-update.
ADR 2 menulis 'Supersedes: 1', tapi ADR 1 tidak pernah dicatat 'Superseded by 2'.
Penomoran loncat: ADR 1, ADR 3 langsung — engineer lupa insert ADR 2.
Status informal seperti 'Draft', 'In Review' diterima tanpa penolakan.

CONTOH KASUS NYATA

ADR 1 ber-status Superseded by 99, padahal ADR 99 belum pernah dibuat.

Ketika tim mengandalkan review manual terhadap dokumen Markdown ADR, pola kegagalan yang paling sering muncul adalah tautan supersesi yang tidak sinkron. Engineer bisa jadi memperbarui isi keputusan pada ADR baru namun lupa mengembalikan marker pada ADR lama. Atau sebaliknya: ADR baru mengklanimsupersedes_ADR lama, tapi ADR lama tidak pernah mengubah statusnya menjadi Superseded. Review manual juga sulit mendeteksi loncatan nomor urut karena tidak ada mekanisme enforcement struktural.

Manual review catches content errors but misses structural relation gaps.

ADR Bukan Dokumen, Tapi Record yang Bisa Dicek

Parser mengubah baris Markdown menjadi struct Record; linter memeriksa aturan struktural.

PRINSIP PENTING

ADR diolah sebagai data, bukan sekadar teks human-readable.
Parser mengekstrak ID, Title, Status, SupersededBy, Supersedes, Content.
Linter memvalidasi aturan: monotonik ID, valid status, relasi timbal-balik.
Keputusan arsitektur tetap ditulis manusiawi; mesin hanya memeriksa format.

Inti dari pendekatan lab ini adalah memandang Architecture Decision Record bukan sebagai dokumen naratif yang perlu dibaca manusia secara manual, melainkan sebagai record terstruktur yang bisa diparse dan divalidasi oleh mesin. Parser membaca Markdown, mengekstrak field-field kunci menjadi struct Record, dan linter kemudian menjalankan serangkaian aturan struktural. Nilai utama bukan pada otomasi pengambilan keputusan, melainkan pada kemampuan mendeteksi anomali struktural yang sering luput dari review manual.

ADR is structured data first, human narrative second; validation catches what humans miss.

Pipeline: Markdown → Parser → Records → Linter

Alur konversi ADR dari teks bebas menjadi record terstruktur dan validasi paralel.

ARCHITECTURE FLOW

```
[Markdown ADR Text] → adr.Parser (Regex extraction: ID, Title, Status, refs)
→ []*adr.Record (In-Memory Structs)
→ adr.Linter.Validate(records)
 ├── Monotonic numbering check: sort + sequential scan
 ├── Cycle detection: DFS with visiting/visited states
 └── Bidirectional lineage: goroutines + sync.Mutex per-record validation
→ []error (empty = valid, non-empty = integrity breach)
```

Arsitektur sistem validasi ADR pada lab ini dibangun sebagai pipeline tiga tahap. Tahap pertama, parser membaca Markdown baris demi baris menggunakan regex dan mengekstrak field-field kunci. Hasilnya adalah slice struct Record yang berisi metadata keputusan. Tahap kedua, linter menjalankan validasi urutan ID monotonik dan deteksi siklus melalui depth-first search. Tahap ketiga, validasi graf silsilah dilakukan secara paralel menggunakan goroutine dengan mutex untuk agregasi error, memastikan setiap tautan SupersededBy dan Supersedes saling konsisten.

Pipeline separates parsing from validation: each stage has a single responsibility.

Struct Record: Definisi Inti ADR

Lima field wajib membuat setiap keputusan arsitektur dapat direferensikan dan diverifikasi.

```
type Status string

const (
    StatusProposed   Status = "Proposed"
    StatusAccepted   Status = "Accepted"
    StatusSuperseded Status = "Superseded"
    StatusDeprecated Status = "Deprecated"
    StatusRejected   Status = "Rejected"
)

type Record struct {
    ID           int
    Title        string
    Status       Status
    SupersededBy int // 0 if not superseded
    Supersedes   int // 0 if does not supersede
    Content      string
}
```

Struktur data Record pada models.go merupakan fondasi seluruh sistem validasi. Field Status menggunakan enum string yang hanya mengakui lima nilai resmi: Proposed, Accepted, Superseded, Deprecated, dan Rejected. Field SupersededBy mereferensikan ADR pengganti, sedangkan Supersedes mereferensikan ADR yang digantikan. Keduanya bernilai 0 (nol) ketika tidak ada relasi. Dengan representasi terstruktur ini, linter tidak perlu menebak-tebak — ia hanya memeriksa konsistensi integer dan string.

Explicit fields over implicit conventions make structural validation possible.

Supersesi Patah: ADR 1 Merujuk ke ADR 99

Tautan Superseded by menunjuk ke nomor yang tidak ada — linter langsung menandai error.

```
superseded points to non-existent:
  ADR 1: Status Superseded by 99
  → Error: superseded by non-existent ADR 99

supersedes points to non-existent:
  ADR 1: Status Accepted, Supersedes: 99
  → Error: supersedes non-existent ADR 99
```

Skenario kegagalan paling umum pada pemeliharaan ADR manual adalah tautan supersesi yang merujuk ke nomor ADR yang belum ada atau sudah dihapus. Linter menangkap kasus ini dengan memverifikasi keberadaan setiap ID referensi di dalam recordMap. Jika ADR 1 menyatakan Superseded by 99 namun tidak ada Record dengan ID 99, pesan error spesifik langsung dihasilkan. Bukti ini berasal dari test case TestLinter_BrokenReferences yang dijalankan secara deterministik.

BUKTI DARI TEST

TestLinter_BrokenReferences / superseded points to non-existent → PASS
TestLinter_BrokenReferences / supersedes points to non-existent → PASS

A dangling reference is cheaper to catch at lint time than during an architectural review.

Biaya Tersembunyi: Lineage Drift Tanpa Deteksi

Setiap ADR baru yang tidak mengikuti aturan relasi meningkatkan kerumunan manual review.

Time to detect	Cause	Impact
Instant (CI)	Linter reject di PR	Build gagal, engineer feedback loop cepat
Mingguan	Manual audit dokumentasi	Engineer kebingungan siapa yang aktif
Bulanan	Production incident review	Tim kehilangan konteks keputusan penting
Selamanya	Keputusan salah diterapkan	Sistem berkembang dengan asumsi keliru

Biaya tersembunyi dari ketiadaan validasi otomatis bukan hanya waktu deteksi, tetapi akumulasi kepercayaan palsu bahwa dokumentasi ADR masih akurat. Setiap kali engineer menambahkan ADR baru tanpa memutakhirkan ADR lama, tercipta discrepancy yang hanya akan terdeteksi saat inspeksi manual. Pada tim kecil, hal ini mungkin tidak terasa. Namun begitu skala tim bertambah, waktu yang terbuang untuk mencari konsistensi manual berlipat ganda tanpa bound yang jelas.

Unverified lineage drift accumulates silently until a production decision depends on it.

Setup Eksperimen: 3 ADR Evolusi Arsitektur

Demo memodelkan transisi dari Modular Monolith ke Microservice dengan satu ADR ditolak.

ADR SEQUENCE

ADR 0001: Use Modular Monolith for Core SaaS ERP | Status: Superseded by 2
ADR 0002: Extract Notification Service to Microservice | Status: Accepted, Supersedes: 1
ADR 0003: Reject Event Sourcing for Order Management | Status: Rejected

Test suite: parser_test.go (valid + superseded + supersedes + invalid cases)
Test suite: linter_test.go (valid sequence + 7 broken reference scenarios + concurrency)

Eksperimen lab ini memodelkan tiga skenario ADR yang merepresentasikan evolusi arsitektur nyata. ADR 1 merekam keputusan awal menggunakan Modular Monolith. ADR 2 menggantikan ADR 1 setelah tim tumbuh melebihi 20 engineer dan membutuhkan scaling independen untuk notification service. ADR 3 menolak usulan Event Sourcing karena kebutuhan audit trail masih terpenuhi oleh relational log table standar. Ketiganya divalidasi bersama melalui linter untuk memastikan konsistensi lineagenya.

Three ADRs capture one realistic architectural evolution path with valid lineage.

Output Demo: Parsing Berhasil, Linter Lolos

Terminal menampilkan 3 ADR terparsing dan integrity check passed tanpa error.

```
$ go run ./cmd/demo
=== Architecture Decision Record (ADR) Lab ===
Successfully parsed 3 ADRs:
  - [0001] Use Modular Monolith for Core SaaS ERP | Status: Superseded -> Superseded by ADR 2
  - [0002] Extract Notification Service to Microservice  | Status: Accepted   -> Supersedes ADR 1
  - [0003] Reject Event Sourcing for Order Management    | Status: Rejected

Running ADR Integrity Linter...
Integrity check passed! Decision lineage and lifecycle invariants are intact.
```

Output demo CLI membuktikan bahwa ketiga ADR berhasil diparse dari string Markdown dan diterjemahkan menjadi struct Record. Lineage chain ADR 1 → ADR 2 terdeteksi sebagai valid: ADR 1 berstatus Superseded dengan reference ke 2, dan ADR 2 berstatus Accepted dengan klaim Supersedes: 1. Linter tidak menemukan error apa pun pada validasi monotonicity, cycle detection, maupun bidirectional link integrity.

Clean linter output confirms structural consistency, not architectural correctness.

Apa Artinya Linter Passed di Lab Ini?

Linter passing membuktikan struktur record konsisten, bukan kebenaran keputusan arsitektur.

YANG DIBUKTIKAN

Format parsing: judul, status, dan tautan supersesi terstruktur dengan benar.
Monotonik: ID 1, 2, 3 berurutan tanpa celah.
Bidirectional: ADR 1 ↔ ADR 2 memiliki tautan timbal-balik yang lengkap.
Cycle-free: tidak ada sirkuit dalam graf supersesi.

YANG TIDAK DIBUKTIKAN

Kebenaran keputusan arsitektur di dalam field Content.
Apakah ADR 1 seharusnya benar-benar digantikan ADR 2 di dunia nyata.
Kesiapan sistem untuk production deployment.

Hasil linter yang passing adalah bukti kondisi necessary, bukan sufficient. Ia membuktikan bahwa ketiga ADR memenuhi invariant struktural: format parsable, nomor urut monotonik, dan tautan supersesi timbal-balik utuh. Namun, linter tidak menilai apakah keputusan Modular Monolith memang tepat, atau apakah penolakan Event Sourcing didasarkan pada analisis yang komprehensif. Kualitas substansi keputusan tetap tanggung jawab review manusia.

Linter passing is necessary but not sufficient for architectural soundness.

Trade-off: Enforced Structure vs Dokumentasi Fleksibel

Standarisasi format ADR meningkatkan deteksi error, tetapi membatasi ekspresi nuansa konteks.

ENFORCED STRUCTURE (LAB INI)

Regex-based parsing memaksa format fixed (## Context, ## Decision, ## Consequences).
Status wajib dari 5 enum resmi; status informal ditolak parser.
Supersedes/SupersededBy wajib konsisten secara timbal-balik.

FLEKSIBILITAS YANG DILEWATI

Tidak mendukung ADR yang memerlukan struktur sub-section kustom di luar 3 section wajib.
Tidak memvalidasi kualitas argumen dalam section Decision.
Tidak menangani kasus ADR multi-author atau decision yang melibatkan beberapa sub-system.

Validasi struktural ADR membawa trade-off mendasar: semakin ketat format yang dipaksakan, semakin rendah fleksibilitas penulisan konteks arsitektur. Lab ini memilih path enforceable machine-checkable, artinya parser akan menolak ADR yang tidak mengikuti skema tetap. Di sisi lain, pendekatan ini memungkinkan CI/CD gate yang andal. Tim yang membutuhkan nuansa lebih kaya bisa menambahkan section tambahan di luar yang diparsing, selama tiga section wajib tetap ada.

Structure enforcement gains verification reliability at the cost of narrative flexibility.

Validasi Paralel: Goroutine + Mutex untuk Agregasi Error

Setiap ADR divalidasi secara independent goroutine, error dikumpulkan dengan sync.Mutex.

```
var wg sync.WaitGroup
for _, r := range records {
    wg.Add(1)
    go func(rec *Record) {
        defer wg.Done()
        var localErrs []error
        // ... validate rec, append localErrs ...
        if len(localErrs) > 0 {
            mu.Lock(); errs = append(errs, localErrs...); mu.Unlock()
        }
    }(r)
}
wg.Wait()
```

Pola validasi konkuren pada linter menunjukkan praktik Go yang tepat untuk parallel error aggregation. Setiap Record divalidasi dalam goroutine terpisah, menghasilkan slice error lokal yang hanyadigabungkan ke global errs slice setelah acquire mutex. Pola ini menghindari race condition pada write ke shared error slice. Meskipun validasi pada 3 record tidak memberikan gain performa signifikan, pola ini dapat diskalakan untuk hundred-an ADR tanpa perubahan struktur kode.

Per-record goroutine with mutex aggregation scales validation without shared-state races.

internal/adr/linter.go:64–106

Before vs After: Otomasi Linter Menghilangkan Human Error

Membandingkan gap deteksi antara review manual dan automated structural linting.

SEBELUM LINTER (MANUAL)

Lineage break: ADR 1 Superseded by 2, ADR 2 tidak memiliki Supersedes: 1 → TIDAK TERDETEKSI
Non-monotonic ID: ADR 1, ADR 3 meloncat → TIDAK TERDETEKSI selagi tidak inspect semua file
Invalid status 'Draft' masuk ke repo → TIDAK TERDETEKSI oleh regex sederhana
Self-supersession: ADR 1 Superseded by 1 → TIDAK TERDETEKSI

SESUDAH LINTER (AUTOMATED)

Semua kasus di atas ditolak oleh TestLinter_BrokenReferences dalam <1ms
CI gate menolak merge jika linter menemukan error
Concurrent stress test membuktikan tidak ada race pada agregasi error

Perbandingan before-after menunjukkan bahwa automasi linter menutup celah deteksi yang secara konsisten terlewatkan pada review manual. Empat skenario failure yang tercatat dalam test suite—dangling reference, mismatched link, non-monotonic ID, dan self-supersession—semuanya lolos dari radar reviewer manusia tetapi langsung tertangkap oleh automated check. Ini bukan berarti linter menggantikan peran arsitek, melainkan mengurangi beban kognitif pada hal-hal yang bersifat mekanis.

Automation closes mechanical review gaps that human reviewers consistently miss.

Batasan Lab: Validasi Format, Bukan Kualitas Keputusan

Linter memeriksa kepatuhan struktur; ia tidak menilai substansi atau kebenaran argumen.

YANG DILAKUKAN LAB

Parsing teks Markdown ke struct Record dengan regex.
Memvalidasi urutan ID monotonik dan uniqueness.
Memeriksa konsistensi timbal-balik SupersededBy ↔ Supersedes.
Mendeteksi siklus dalam graf supersesi.

YANG TIDAK DILAKUKAN LAB

Tidak menilai kualitas argumen di dalam section Decision.
Tidak memverifikasi apakah ADR cocok dengan implementasi kode aktual.
Tidak memeriksa apakah engineer sudah mendapatkan approval stakeholder sebelum Accepted.
Simulasi ini tidak sama dengan production ADR management system seperti ADRtool atau Architect.js.

Batasan paling penting dari lab ini adalah ia hanya memeriksa kepatuhan format, bukan kebenaran konten. Parser tidak tahu apakah keputusan 'Extract Notification Service to Microservice' adalah keputusan yang tepat—ia hanya memverifikasi bahwa keputusan tersebut ditulis dalam format yang benar dengan relasi supersesi yang konsisten. Klaim validasi otomatis ini berlaku pada jalur yang diuji, bukan jaminan bahwa seluruh ekosistem dokumentasi arsitektur tim sudah bersih.

Format validation ensures structural integrity; it does not evaluate architectural soundness.

Edge Case: Keputusan Dibatalkan Tanpa ADR Baru

Engineer menghentikan fitur tanpa membuat ADR Deprecated; state sistem menjadi ambigu.

SKENARIO EDGE CASE

ADR X berstatus Accepted, tapi tim memutuskan berhenti menggunakan fitur tersebut.
Tanpa ADR baru, status ADR X tetap Accepted —误导 engineer lain以为 masih relevan.
Solusi: buat ADR baru berstatus Deprecated merujuk ke alasan penghentian.
Kegagalan: engineer menghapus file ADR langsung → lineage history hilang.

REKOMENDASI PRODUKSI

Wajibkan policy tim: setiap penghentian fitur memerlukan ADR Deprecated baru.
Linter bisa diperluas untuk mendeteksi ADR Accepted yang tidak memiliki successor dalam 12 bulan.

Kasus tepi yang tidak ditangani oleh linter saat ini adalah penghentian keputusan tanpa dokumentasi formal. Ketika engineer secara diametral menghentikan fitur yang sebelumnya di-accept via ADR, status dokumen tetap Accepted meskipun praktik aktual sudah berubah. Ini menciptakan gap antara dokumentasi dan realitas sistem. Solusi yang direkomendasikan adalah policy tim yang mewajibkan ADR Deprecated sebagai pengganti, bukan penghapusan file.

Silent feature deletion without Deprecated ADR creates documentation-reality gap.

Apa yang Dibuktikan Oleh Test Suite

Parser dan linter menutupi semua skenario struktural dasar yang didefinisikan dalam lab.

PARSER TESTS (PARSER_TEST.GO)

TestParse_Valid: ID=1, Status=Accepted → PASS
TestParse_Superseded: SupersededBy=2 berhasil diekstrak → PASS
TestParse_Supersedes: Supersedes=1 berhasil diekstrak → PASS
TestParse_Invalid: missing_title, missing_status, invalid_status, empty sections → semua PASS

LINTER TESTS (LINTER_TEST.GO)

TestLinter_ValidSequence: 3 ADR valid, zero errors → PASS
TestLinter_BrokenReferences: 7 sub-case (non-existent ref, mismatch, non-monotonic, self-super, duplicate ID, cyclical) → semua PASS
TestLinter_ConcurrencyStress: concurrent validation, no data race → PASS

Suite pengujian lab ini mencakup 10 test function dengan 18 sub-case yang membuktikan seluruh invariant struktural yang didefinisikan. Parser menangani 6 kasus validasi format (termasuk 9 sub-case invalid). Linter menangani 8 skenario integritas graf (valid sequence, 7 broken reference types). Race detector pada go test -race juga lulus tanpa warning. Bukti ini cakupannya mengikuti skenario test yang didefinisikan — bukan klaim bahwa seluruh kemungkinan edge case production tertutup.

Test coverage proves structural invariants on defined scenarios, not all possible production states.

Apa yang Tidak Dibuktikan Oleh Test Suite

Test hanya menjangkau jalur yang di-skripsikan; production masih perlu validasi tambahan.

BUKAN BUKNIT TEST

Kebenaran konten arsitektur di dalam section Decision dan Consequences.
Adopsi ADR oleh seluruh anggota tim sesuai format yang ditetapkan.
Integrasi linter dengan CI/CD pipeline (hanya test Go murni, bukan GitHub Actions).
Skalabilitas linter pada ribuan ADR (hanya 3 record dalam demo, 3–7 dalam test).

SIMULASI IN-MEMORY

Parser bekerja pada string literal, bukan file system read dari ribuan dokumen Markdown.

Penting untuk mengenali batas bukti dari test suite ini. Test tidak menilai kebenaran substansi keputusan arsitektur yang terdapat di dalam field Content. Test juga tidak membuktikan bahwa linter akan bekerja dengan baik ketika diterapkan pada ratusan file ADR di repositori production. Simulasi berjalan pada string literal di dalam kode, bukan pada file system I/O aktual. Klaim validasi struktural yang dihasilkan test berlaku pada jalur yang diuji, bukan generalisasi ke seluruh spektrum kemungkinan error di lapangan.

Unit tests prove behavior on defined inputs; they do not guarantee production behavior.

Risiko Produksi: False Confidence dari Linter Hijau

Linter passing bisa memberikan ilusi keamanan yang menyesatkan jika substansi tidak dicek manusia.

FALSE POSITIVE RISKS

Tim percaya ADR collection-nya 'bersih' hanya karena linter passing, padahal argumen Decision lemah.
Engineer bisa mengisi Content dengan placeholder kosong selama section ada → linter tetap passing.
Policy tim yang tidak enforcing bisa di-skip tanpa konsekuensi jika gate CI tidak dipasang.

MITIGASI

Tambah human review checkpoint: ADR Accepted wajib ditinjau architecture board.
Validasi minimum content length pada section Context dan Decision.

Risiko terbesar dari automasi validasi adalah false confidence. Ketika linter menghasilkan output hijau tanpa error, tim cenderung menganggap seluruh aspek dokumentasi arsitektur sudah OK. Padahal, Content field bisa saja berisi penjelasan yang sangat minim atau bahkan placeholder. Linter passing bukan berarti keputusan arsitektur tersebut berkualitas tinggi — ia hanya berarti formatnya konsisten secara struktural. Mitigasi utamanya adalah mempertahankan human review sebagai gate tambahan.

A green linter pass validates structure, not wisdom. Human review remains irreplaceable.

Implementation Checklist: Menerapkan ADR Linter di Tim

Langkah-langkah praktis untuk mengintegrasikan parser-linter ke workflow tim.

- ✓ [ ] Define ADR template Markdown dengan 3 section wajib: Context, Decision, Consequences.
- ✓ [ ] Implement parser regex yang mengekstrak ID, Title, Status, SupersededBy, Supersedes.
- ✓ [ ] Tambahkan status enum valid (Proposed, Accepted, Superseded, Deprecated, Rejected).
- ✓ [ ] Implement linter dengan validasi monotonik + bidirectional supersession + cycle detection.
- ✓ [ ] Integrasikan ke CI/CD: go test ./... atau custom binary di PR gate.
- ✓ [ ] Dokumentasikan policy tim: setiap perubahan keputusan wajib ADR baru, bukan edit retroaktif.

Checklist ensures consistent adoption; culture change ensures sustained compliance.

Anti-Pattern: 5 Kebiasaan Buruk Saat Mengadopsi ADR

Kesalahan umum yang mengurangi efektivitas ADR dan cara menghindarinya.

ANTI-PATTERN SERING TERJADI

Mengklik ADR untuk segala hal: refactor kecil, penamaan variabel → noise berlebihan.
Mengedit ADR yang sudah Accepted secara silent, tanpa membuat ADR baru (retroactive edit).
Menulis ADR hanya sebagai catatan meeting tanpa section Decision yang eksplisit.
Status 'Draft' bertahan terlalu lama tanpa transisi ke Accepted atau Rejected.
Melupakan update SupersededBy pada ADR lama saat ADR baru dibuat.

Lima anti-pattern ini sering muncul saat tim pertama kali mengadopsi ADR. Pola yang paling merusak adalah retroactive edit — suntingan diametral pada ADR yang sudah Accepted. Ini menghancurkan jejak keputusan dan membuat engineer berikutnya membaca konteks yang sudah tidak relevan. ADR linter lab ini dirancang untuk mencegah sebagian besar anti-pattern tersebut secara struktural, meskipun tidak menutupi masalah adopsi budaya tim.

ADR anti-patterns are cultural; linter catches structural symptoms, not behavioral roots.

Reproduksi Mandiri: Jalankan Demo dan Test di Lokal

Semua dependency menggunakan Go standard library, tanpa framework eksternal.

```
cd labs/17-architecture-decision-record
go test -v ./...
go test -race ./...
go run ./cmd/demo
```

EXPECTED OUTPUT

TestParse_*: semua PASS
TestLinter_*: semua PASS (14 sub-case total)
go test -race: race detector clean
Demo: 3 ADR parsed, Integrity check passed

Laboratorium ini dirancang untuk reproduksi instan di mesin manapun yang menjalankan Go 1.22+. Tidak ada dependensi eksternal, tidak ada database, tidak ada network call. Command go run ./cmd/demo menjalankan demo CLI yang memuat tiga ADR dari string literal, mem-parsenya, dan menjalankan linter. Command go test -race memastikan tidak ada race condition pada konkurensi linter. Seluruh artefak testing dan catatan audit tersedia di direktori lab untuk studi lebih lanjut.

Zero-dependency Go standard library ensures instant reproduction on any machine.

ADR Diubah Jadi Data yang Bisa Dicek, Bukan Hanya Dokumen

Parser+linter memindahkan keputusan arsitektur dari domain narasi ke domain struktur terverifikasi.

PELAJARAN REKAYASA

ADR yang diparse menjadi struct Record memungkinkan validasi otomatis yang tidak bisa dilakukan humans dengan andal.
Bidirectional supersession link adalah invarian yang paling sering rusak; linter menangkapnya di detik pertama.
Konkurensi goroutine + mutex pattern untuk error aggregation adalah reusable pattern untuk validation framework lain.
Format standar membantu review; kualitas keputusan tetap ditentukan isi ADR.

Pelajaran rekayasa perangkat lunak utama dari lab ini adalah pergeseran perspektif: dari ADR sebagai dokumen naratif statis menjadi ADR sebagai record terstruktur yang dapat diverifikasi mesin. Parser dan linter tidak menggantikan judgment arsitek, tetapi mereka menutupi gap mekanisme yang konsisten terlewat oleh review manual. Pola validasi konkuren yang diterapkan di linter juga bersifat generik dan dapat diadaptasi untuk validasi dokumen struktural lain di luar konteks ADR.

Structured validation catches what human review consistently misses: mechanical relation gaps.

Parser membuat ADR terstruktur; linter baru memeriksa aturannya.

Parser makes ADR machine-readable; linter enforces structural invariants. Human wisdom remains irreplaceable.

Software Engineering Lab 17 – Architecture Decision Record

internal/adr/models.go • internal/adr/parser.go • internal/adr/linter.go
cmd/demo/main.go • tests/parser_test.go • tests/linter_test.go
README.md • content/02-master-draft.md

REPOSITORY

`github.com/lukman-ss/software-engineering-lab/tree/main/labs/17-architecture-decision-record`

Lukman / lukman-ss

Sumber: commit def0f5a
Test passing: 14 test cases, 0 failures.
Race detector: clean (go test -race ./...)
Simulasi in-memory string parsing; bukan proof production ADR governance completeness.